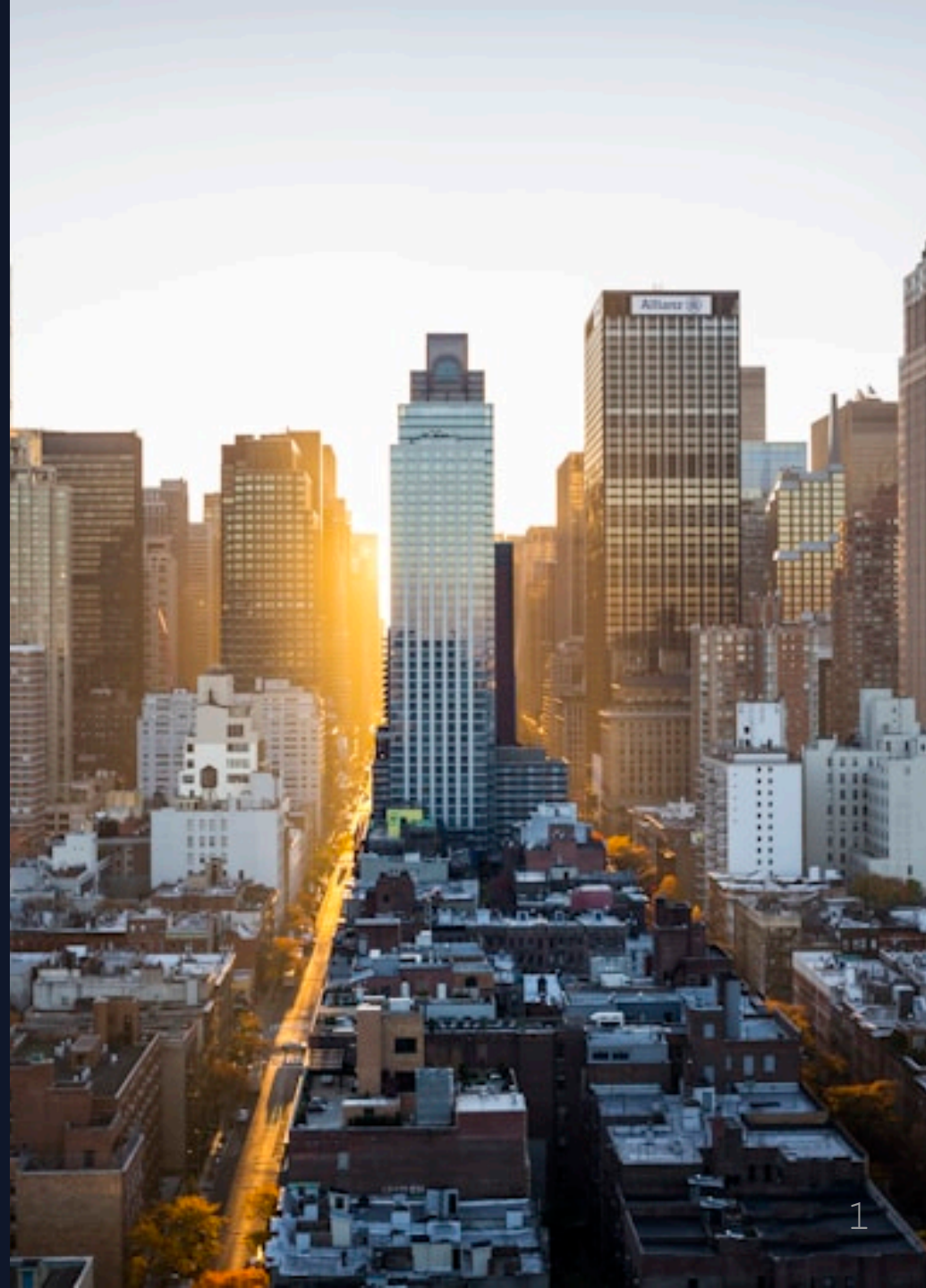
L17: PROJECT - Grid Defense

Procedural Generation & Urban Environment

Neon City Cyber Academy

Module 4: Visual Algorithms



👥 Your Team

Emily Chen - *Handler*

> "Your final test: Build a laser grid defense system. Procedurally generated. Scalable. Pro-level code."

Cole Martinez - *Tech Lead*

> "Procedural generation means: Write a function ONCE, use it FOREVER. This is how cities are built in games."

🎯 Project Brief: Grid Defense

Mission: Protect the Neon City server with a laser grid

Requirements:

1. Generate grid of laser towers
2. Calculate optimal spacing
3. Verify complete coverage
4. Output defense coordinates

Constraints: Text output only (Judge0 compatible)

Procedural Generation Concept

Manual approach (BAD):

```
print("Tower at (0, 0)")
print("Tower at (100, 0)")
print("Tower at (200, 0)")
# ... 97 more times
```

Procedural approach (GOOD):

```
for x in range(0, 1000, 100):
    print(f"Tower at ({x}, 0)")
```

THIS is procedural generation!

The Laser Grid

6x4 grid = 24 towers

💬 Emily's Grid Formula:

> "Two nested loops = 2D grid"

```
# Generate 3x3 grid
spacing = 100

for row in range(3):
    for col in range(3):
        x = col * spacing
        y = row * spacing
        print(f"Tower at ({x}, {y})")
```

Output: 9 tower positions!

🔢 Grid Generation Code

```
def generate_grid(width, height, spacing):
    """Generate defense grid coordinates"""
    towers = []

    rows = height // spacing + 1
    cols = width // spacing + 1

    for row in range(rows):
        for col in range(cols):
            x = col * spacing
            y = row * spacing
            towers.append((x, y))

    return towers
```

🎯 Using the Function

```
# Generate grid for 300x200 area, 100-unit spacing
grid = generate_grid(300, 200, 100)

print(f"Total towers: {len(grid)}")
for i, (x, y) in enumerate(grid, 1):
    print(f"Tower {i}: ({x}, {y})")
```

Output:

```
Total towers: 12
Tower 1: (0, 0)
Tower 2: (100, 0)
Tower 3: (200, 0)
...
```

💬 Cole's Optimization Tip:

> "Calculate BEFORE generating:"

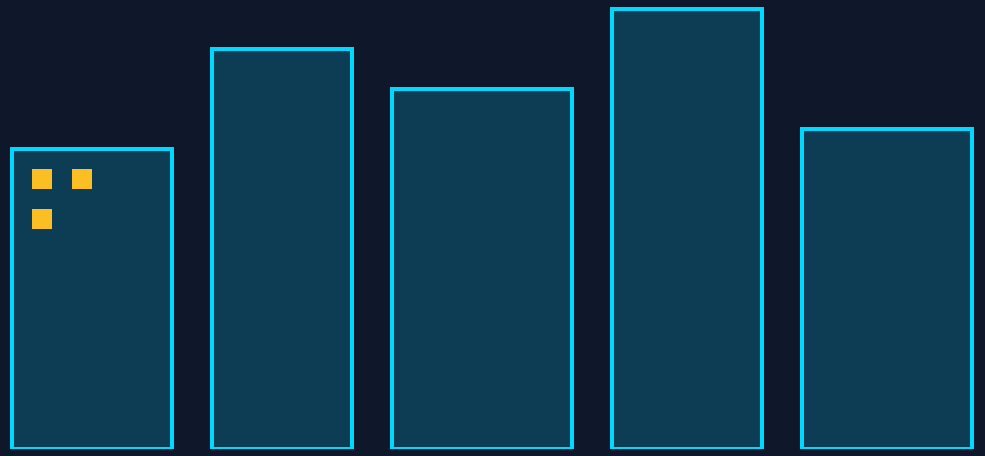
```
width, height = 500, 400
spacing = 50

# Calculate grid dimensions
cols = (width // spacing) + 1   # 11 columns
rows = (height // spacing) + 1  # 9 rows
total = cols * rows              # 99 towers

print(f"Grid will need {total} towers")
```

> "Always estimate resources BEFORE building!"

🏙️ City Skyline Generator



Procedurally Generated

🎨 Skyline Code

```
import random

def generate_skyline(num_buildings):
    """Generate city skyline coordinates"""
    x = 0
    for i in range(num_buildings):
        width = random.randint(50, 100)
        height = random.randint(100, 250)
        print(f"Building {i+1}: x={x}, w={width}, h={height}")
        x += width + 20  # 20-unit gap

generate_skyline(5)
```

🔐 Defense Tower Function

```
def draw_defense_tower(height):
    """Print ASCII art of defense tower"""
    # Top
    print("  /\\")

    # Body (based on height)
    for i in range(height):
        print(" |  |")

    # Base
    print("|____|")

# Test
draw_defense_tower(3)
```

Output:

```
  /\
 |  |
 |  |
 |  |
|____|
```

💬 Emily's Design Pattern:

> "Procedural generation follows a template:"

1. **Define parameters** (width, height, spacing)
2. **Calculate dimensions** (rows, cols, total)
3. **Loop through grid** (nested for loops)
4. **Generate each element** (position, size, etc.)
5. **Store or print result**

> "This pattern generates EVERYTHING: Cities, dungeons, forests, space!"

🎯 Coverage Verification

```python
def verify_coverage(grid, target_width, target_height, laser_range):
    """Check if grid covers entire area"""
    # For each tower, check if it reaches boundaries
    max_x = max(x for x, y in grid)
    max_y = max(y for x, y in grid)

    covered_x = max_x + laser_range >= target_width
    covered_y = max_y + laser_range >= target_height

    return covered_x and covered_y

# Test
grid = [(0,0), (100,0), (200,0)]
print(verify_coverage(grid, 250, 100, 50))  # True
```

🏆 Final Project Challenge

Build a complete Grid Defense System:

1. Function: `generate_defense_grid(width, height, spacing)`
2. Calculate total towers needed
3. Generate all tower coordinates
4. Print formatted output
5. Verify 100% coverage

Bonus: Add tower IDs (ALPHA-001, ALPHA-002...)

✅ Module Complete!

You Mastered (L14-L17):

- ✅ Cartesian coordinates & navigation
- ✅ Distance calculations (Pythagorean)
- ✅ Pattern efficiency with loops
- ✅ Boundary detection & collision
- ✅ Procedural generation algorithms
- ✅ Function-based design

Real-World Skills Gained:

🎮 Game development basics

🤖 Robotics navigation

📊 Data visualization

Congratulations, Agent!

You've completed the **Drone Pathfinding Module**!

Next: Module 5 - Object-Oriented Programming

Learn: Classes, objects, methods, and advanced Python